SOFTWARE REQUIREMENT SPECIFICATION
Version 1.0

May 2026

KABAKAL GYM MANAGEMENT AND ANALYTICS WEBSITE

BY:
Anna Erika Alonzo
Ceasar Ivan Caiga
Marc Arthur Clarete
Don Noriesta
Symphony Ashley Pingol
Desirie Zy Sanchez
Tristan Jhoeone Talavera

Submitted in partial fulfillment
Of the requirements of
ELEC IT-FE2: BSIT Free Elective 2

TABLE OF CONTENTS

LIST OF FIGURES.....3
1.0. INTRODUCTION.....4
1.1. Purpose.....4
1.2. Scope of the Project.....4
1.3. Glossary.....5
1.4. References.....6
1.5. Overview of Document.....7
2.0. OVERALL DESCRIPTION.....8
2.1. System Environment.....8
Figure 1 - Kabakal Gym System Architecture.....8
2.2. Functional Requirements Specification.....9
2.2.1 Member Use Cases.....9
2.3 User Characteristics.....14
2.4 Non-Functional Requirements.....15
3.0. REQUIREMENTS SPECIFICATION.....15
3.1. External Interface Requirements.....15
3.2. Functional Requirements.....15
3.2.1 Register / Login Functionality.....16
3.2.2 Purchase Subscription.....16
3.2.3 Scan Check-In (QR).....17
3.2.4 Generate Workout.....18
3.2.5 View Analytics Dashboard.....18
3.2.6 Manage Members.....19
3.2.7 Manage Equipment.....20
3.3 Detailed Non-Functional Requirements.....21
3.3.1 Logical Structure of the Data.....21
A. Entity-Relationship Diagram (ERD).....21
Figure 2 - Kabakal Gym Entity-Relationship Diagram.....21
3.3.2 Security.....28
INDEX.....29

LIST OF FIGURES

Figure 1 - Kabakal Gym System Architecture..... 7

Figure 2 - Kabakal Gym Entity-Relationship Diagram..... 18

1.0. INTRODUCTION

1.1. Purpose

The purpose of this document is to present a detailed description of the Kabakal Gym Management and Analytics Website. This document will explain the purpose and features of the system, the interfaces of the system, what the system will do, the constraints under which it must operate and how the system will react to external stimuli. Moreover, this document is intended for both the stakeholders and the developers of the system and will be proposed to our dearest professor Ma. Michaela C. Alejandria for the system's approval.

1.2. Scope of the Project

This software system is a web-based application designed to digitize and streamline the operations of Kabakal Gym. It transitions the gym from manual pen-and-paper logs to an automated, centralized platform.

The system maximizes the gym owner's efficiency by providing automated attendance tracking via QR codes, real-time subscription management, and a Business Analytics Dashboard to monitor revenue and peak facility hours. For gym members, the system acts as a self-service portal where they can securely purchase memberships (e.g., ₱50 Day Pass, ₱699 Monthly Pass) and use an Automated Workout Generator to receive daily routines based on established fitness splits (e.g., PPL, Full Body).

1.3. Glossary

Term	Definition
Admin / Gym Owner	The user with full access to the system, capable of managing members, viewing analytics, and modifying facility data.
Data	Raw facts and figures that are unprocessed.
Database	A storage for all the data collected by an application or program.
Developer	A person who creates new products in software.
Geofencing	A location-based security measure using the device's GPS to ensure a member is physically at the gym when scanning the attendance QR code.
Landing Page	The page customers are taken to when they click a link or search for a website.
Member / Customer	A registered user of the gym who utilizes the web app to pay for subscriptions, generate workouts, and log attendance.
N-Tier Architecture	Divides an application into logical layers and physical tiers.
Portal	A page on the internet that allows people who are interested in a particular subject, to get useful information and to find other websites.

QR Code	A pattern of black and white squares that takes you to a link or encodes hidden messages.
RBAC (Role-Based Access Control)	A security mechanism that restricts system access based on the user's assigned role (Admin vs. Member).
Software Requirements Specification	A document that defines all the functions and uses of the proposed project.
Stakeholder	Any individual that is involved in the project but not one of the developers.
User	The individual who uses the web application, and can be either an admin or customer.
Workout Split	A specific training schedule that divides workout sessions by muscle groups or movement types (e.g., Bro Split, Push/Pull/Legs).

1.4. References

Cambridge Dictionary. (n.d.). *Database*.
https://dictionary.cambridge.org/us/dictionary/english/database

Cambridge Dictionary. (n.d.). *Developer*.
https://dictionary.cambridge.org/us/dictionary/english/developer

1.5. Overview of Document

The next chapter, the Overall Description section, of this document will give an overview of the functionality of the product. It will describe the informal requirements and is used to establish a context for the technical requirements specification in the next chapter.

The third chapter, Requirements Specification section, of this document is written primarily for the developers and describes in technical terms the details of the functionality of the product.

Both sections of the document will describe the same software product in its entirety, but are intended for different audiences and thus use different language.

2.0. OVERALL DESCRIPTION

2.1. System Environment

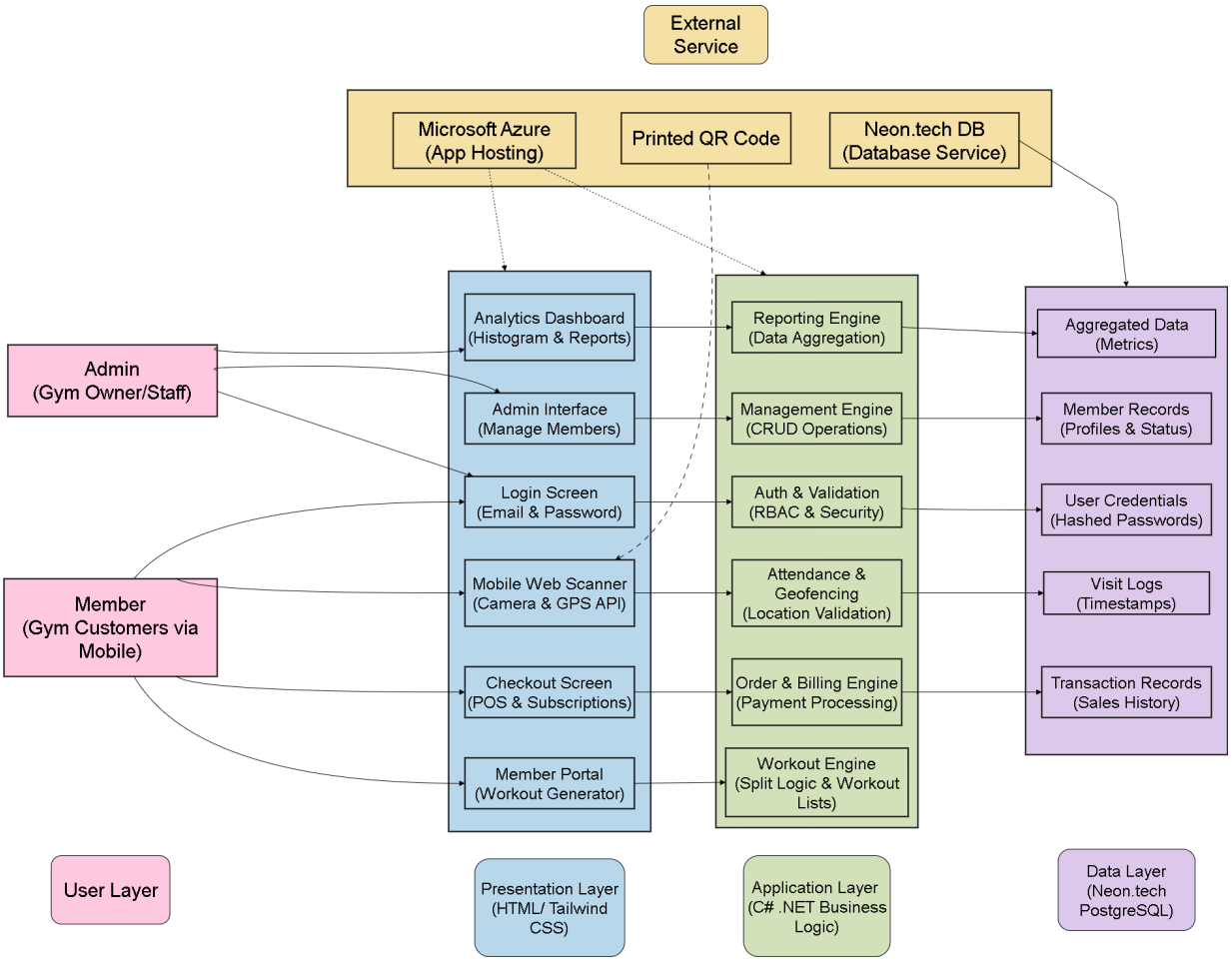


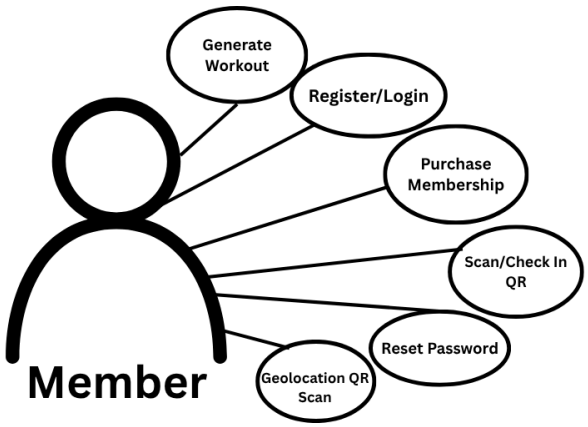
Figure 1 - Kabakal Gym System Architecture

The Kabakal Gym Management and Analytics WebApp utilizes a cloud-hosted uniform architecture designed to serve two active user roles: Members and Administrators. The system cooperates with an external cloud hosting environment (Microsoft Azure) and a cloud database (Neon.tech PostgreSQL).

Members access the self-service portal via mobile web browsers on their smartphones. This allows them to conveniently track workouts, purchase subscriptions, and perform GPS-validated QR code scanning while on the gym floor. In contrast, Administrators use desktop browsers to access the centralized Gym Administration Hub. From this dashboard, the Admin can run CRUD operations on members, equipment records, track real-time revenue, and analyze facility trends.

2.2. Functional Requirements Specification

2.2.1 Member Use Cases



Use Case 1: Register / Login

Brief Description:

The member creates a secure account or logs in using their email and password.

Initial Step-by-step Description:

Before this use case can be initiated, the Member has already connected to the Web Application.

1. The Member creates an account with their own email and password, if they're new to the web application.
2. The Member logs in to the web application using their registered email and password.
3. The Member is welcomed at the home page of the web application after.

Use Case 2: Purchase Subscription

Brief Description:

The member uses the E-Commerce portal to purchase or renew a Day Pass or Monthly Pass.

Initial step-by-step Description:

Before this use case can be initiated, the Member has already accessed the Web Application and has proceeded with the E-Commerce portal.

1. The Member selects the E-Commerce portal.
2. The Member chooses either to purchase a Pass, or renew a Pass.
3. If a Pass is selected, the Member decides if he/she should purchase or renew a Day Pass, or a Monthly Pass.
4. The Member pays for that Pass.
5. The system notifies the Member a purchase receipt which includes the purchased pass and date of purchase.

Use Case 3: Scan Check-In (QR)

Brief Description:

The member uses their mobile device camera to scan the gym's printed QR code, validating their location via GPS to log their attendance.

Initial step-by-step Description:

Before this use case can be initiated, the Member must be logged in to the web application, and has already opened their own mobile device camera, or a QR code scanner application.

1. The Member opens their mobile device camera, or QR code scanner application.
2. The Member scans the gym’s printed QR code.
3. The Member is directed to the web application to validate their location.
4. Once location is validated, the Member’s attendance is automatically recorded.

Use Case 4: Generate Workout

Brief Description:

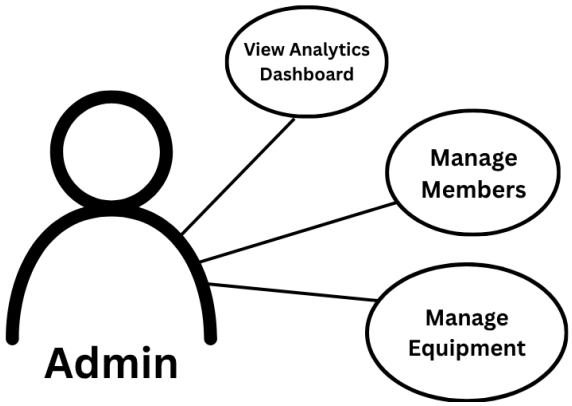
The member selects a preferred training split, and the system outputs a structured daily routine.

Initial step-by-step Description:

Before this use case can be initiated, the Member has already accessed the Web Application after the *Register / Login* use case.

1. The Member selects their preferred training split.
2. The system processes the chosen training split, and displays the corresponding workout plan.
3. The system generates a structured daily routine, including exercises, sets, and repetitions.
4. The Member reviews the generated workout routine.
5. The Member saves or proceeds to follow the displayed workout plan.

2.2.2 Admin Use Cases



Use Case 1: View Analytics Dashboard

Brief Description:

The Admin views visual representations of gym revenue, peak hour histograms, and member churn rates.

Initial step-by-step Description:

Before this use case can be initiated, the Admin has already access to the Web Application and proceeds to the analytics dashboard.

1. The Admin selects the analytics dashboard.
2. The Admin reviews key metrics such as total revenue, peak usage hours, and member churn rate.
3. The Admin filters or adjusts the displayed data based on date range or specific criteria.
4. The Admin interprets the visual data to support decision-making.

Use Case 2: Manage Members

Brief Description:

The Admin performs CRUD operations to update member statuses, edit details, or manually override subscription dates.

Initial step-by-step Description:

Before this use case can be initiated, the Admin has already logged into the web application with valid credentials and has appropriate access rights to manage member records, and ensure every existing member data is available in the database.

1. The Admin navigates to the “Manage Members” section of the system.
2. The Admin selects a specific member record to view or modify.
3. The Admin performs the required action (create, read, update, or delete member details / statuses / subscription dates).
4. The system automatically saves the changes and updates the member record accordingly.

Use Case 3: Manage Equipment

Brief Description:

The Admin logs new equipment purchases or schedules maintenance.

Initial step-by-step Description:

Before this use case can be initiated, the Admin has already accessed the “management dashboard” and has the necessary permissions to manage equipment records.

1. The Admin navigates to the “Manage Equipment” section of the system.
2. The Admin selects whether to add new equipment or update an existing equipment record.
3. The Admin enters equipment details or schedules maintenance as needed.
4. The system automatically saves the information and updates the equipment records accordingly.

2.3 User Characteristics

The system supports two distinct user roles with the following characteristics and technical proficiencies:

Member(s)

Characteristics: Registered gym customers utilizing the web app to pay for subscriptions, generate workouts, and log attendance.

Technical Proficiencies: Expected to be smartphone-literate, familiar with basic web navigation, and capable of granting browser camera and location permissions when prompted during the check-in process.

Admin(s)

Characteristics: The gym owner or authorized personnel with full access to the system. Capable of modifying facility data, tracking revenue, and updating equipment records.

Technical Proficiencies: Expected to be desktop and smartphone-literate, with the ability to read and interpret business intelligence data charts, usage histograms, and digital management ledgers.

2.4 Non-Functional Requirements

To ensure secure, reliable operations, the application is governed by the following technical standards.

The webApp mainly utilizes a monolithic architecture hosted on Microsoft Azure with the backend is built using C# .NET, utilizing Entity Framework Core to connect with a Neon.tech PostgreSQL cloud database. The frontend is to be built using HTML and Tailwind CSS to ensure complete responsiveness across mobile devices and desktop views. System pages, especially workout routines and the analytics dashboard, are expected to load in under 5 seconds under normal operational conditions.

All user credentials are cryptographically hashed using the .NET Identity Framework, where the system enforces Role-based Access Control (RBAC) at the application layer to prevent unauthorized access to restricted API endpoints.

3.0. REQUIREMENTS SPECIFICATION

3.1. External Interface Requirements

The system interacts with the Browser Geolocation API on the user's mobile device to capture latitude and longitude coordinates during the QR Check-in process. This ensures the user is within the physical perimeter of Kabakal Gym.

3.2. Functional Requirements

The Logical Structure of the Data is contained in Section 3.3.1.

3.2.1 Register / Login Functionality

Use Case Name	**Register / Login**
Trigger	The Member registers, or logs in to the system.
Precondition	The Member is connected to the system.
Basic Path	1. The system presents the register / login menu. 2. The Member inputs email and password. 3. The system confirms their login details. 4. The system logs them in. 4. The Member is now welcomed at the home page of the web application.
Alternative Paths	If the password entered is incorrect, the system alerts the user and returns them to step 2.
Postcondition	The web application is accessible.

3.2.2 Purchase Subscription

Use Case Name	**Purchase Subscription**
Trigger	The Member selects "Renew Pass" or "Buy Pass" from their dashboard.
Precondition	The Member is logged into the system and their account is active.
Basic Path	1. The system presents the available pricing tiers (₱50 Day Pass, ₱699 Monthly Pass). 2. The Member selects a tier. 3. The system presents the checkout/payment gateway. 4. The Member inputs payment details and submits. 5. The system processes the payment, generates a Transaction Record, and updates the Member's SubscriptionEndDate.
Alternative Paths	If the payment is declined or fails, the system alerts the user and returns them to step 3.
Postcondition	The database is updated with the new expiration date and financial transaction.

3.2.3 Scan Check-In (QR)

Use Case Name	**Scan Check-In (QR)**
Trigger	The Member taps the "Check-In" button on the mobile portal.
Precondition	The Member is physically at the gym and logged into the web app.

Basic Path	1. The system prompts the user to allow Camera and Location permissions. 2. The Member scans the physical QR code at the gym desk. 3. The frontend captures the QR data and current GPS coordinates, sending them to the C# Backend. 4. The Backend verifies the coordinates are within 50 meters of the gym's official location. 5. The system creates a Visit record with the current timestamp. 6. The system displays a "Check-in Successful" confirmation.
Alternative Paths	In step 4, if the GPS coordinates are outside the geofenced area, the system rejects the check-in and displays an error ("You must be at the gym to check in").
Postcondition	A new attendance timestamp is logged in the database for analytics.

3.2.4 Generate Workout

Use Case Name	**Generate Workout**
Trigger	The Member navigates to the Workout Engine tab.
Precondition	The Member has an active, paid subscription.
Basic Path	1. The system displays available training splits (Bro Split, PPL, Full Body). 2. The Member selects their desired split. 3. The backend C# logic queries the Workout DB for exercises matching the selected tags. 4. The system aggregates a structured list of exercises and returns it to the frontends. 5. The Member views their daily routine.

Alternative Paths	If the member selects an advanced split but their profile is marked as "Beginner," the system prompts them with a warning before generating the routine.
Postcondition	A custom workout view is rendered for the user.

3.2.5 View Analytics Dashboard

Use Case Name	**View Analytics Dashboard**
Trigger	The Admin logs into the system.
Precondition	The user must be authenticated with the Admin role.
Basic Path	1. The system authenticates the user's RBAC status. 2. The backend Reporting Engine runs LINQ queries aggregating data from Transactions, Members, and Visits. 3. The system renders the dashboard displaying Total Revenue, Active Members, and Peak Hour Histograms.
Alternative Paths	In step 1, if a standard Member attempts to access the URL, the system redirects them to the Member Portal.
Postcondition	Accurate business intelligence metrics are displayed.

3.2.6 Manage Members

Use Case Name	**Manage Members**
Trigger	The Admin logs into the system and navigates to the Member Management page.
Precondition	The user must be authenticated with the Admin role, and the member’s data must already exist in the database.

Basic Path	1. The system displays list/table of all active and inactive member accounts 2. The Admin selects a member record to view their profile, subscription status, or visit history. 3. The backend Cloud PostgreSQL allows the Admin to perform CRUD operations on the selected record: ● Create: Add a new member account. ● Read: View specific subscription and attendance information. ● Update: Edit account details or update subscription expiration dates. ● Delete: Remove a member profile from the database. 4. The system updates the database and displays a success message.
Alternative Paths	If the member's data record does not exist, the system alerts the Admin that the "Data does not exist" and returns to step 2. And if an unauthorized or non-Admin user attempts to access the management URl, the system redirects them to the Member Portal.
Postcondition	The member's data record, and the whole record is updated.

3.2.7 Manage Equipment

Use Case Name	**Manage Equipment**
Trigger	The Admin logs into the system and navigates to the Equipment Management section.
Precondition	The user must be authenticated with the Admin role, and must have necessary permissions to manage equipment records.

Basic Path	1. The system displays all equipment records. 2. The Admin selects equipment and updates its record. 3. The system saves the changes, and the exercise belongs to an equipment is available.
Alternative Paths	If the equipment is unavailable, the exercise is out of service, and returns to step 2.
Postcondition	The equipment record is updated.

3.3 Detailed Non-Functional Requirements

3.3.1 Logical Structure of the Data

A. Entity-Relationship Diagram (ERD)

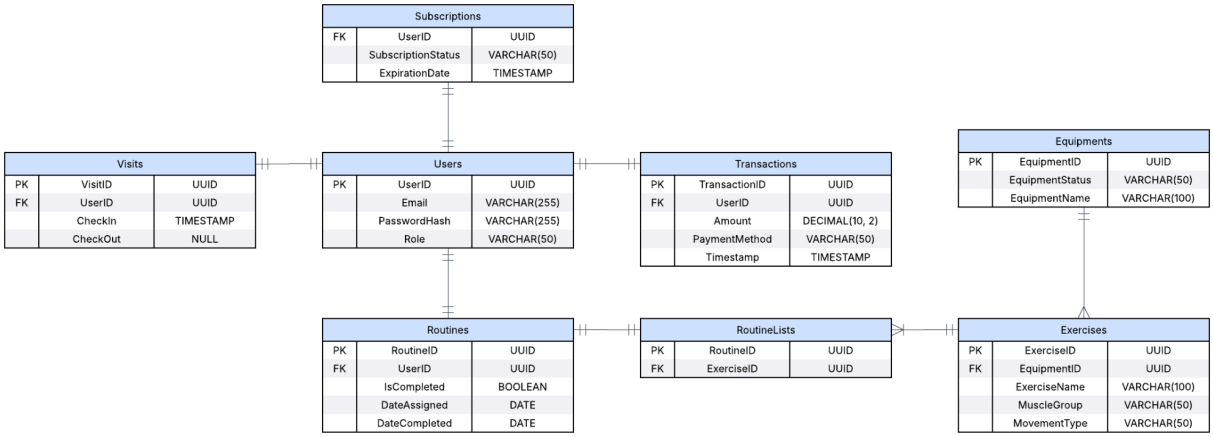


Figure 2 - Kabakal Gym Entity-Relationship Diagram

B. Data Dictionary

The database is structured in PostgreSQL (Neon.tech) using the following tables and definitions:

Table 1: Users (Stores user credentials and roles)

Field Name	Data Type	Constraint	Description	Comment
UserId	**UUID**	Primary Key	Unique identifier for the user's account.	Used as a foreign key in Subscriptions, Transactions,

				Visits, and Routines tables.
Email	**VARCHAR(255)**	Unique, Not Null	The user's login email.	Used for system authentication.
PasswordHash	**VARCHAR(255)**	Not Null	Cryptographically hashed password.	Never stored in plain text to ensure data security.
Role	**VARCHAR(50)**	Not Null	Defines access level (Admin or Member).	Utilized by the Role-Based Access Control (RBAC) engine.

Table 2: Subscriptions (Stores user payment status and subscription status)

Field Name	Data Type	Constraint	Description	Comment
UserID	**UUID**	Primary Key, Foreign Key	References the members in Users table.	Identifies which user the subscription info is referred to.
PaymentStatus	**VARCHAR(50)**	Not Null	Identifies user's current status (Paid, Unpaid, Pending).	Checked before allowing gym entry or generating workout routines.

ExpirationDate	**TIMESTAMP**	Nullable	The date and time the user's access expires.	Automatically triggers a 'past due' alert for the Admin when the date passes.

Table 3: Transactions (Stores financial ledger data for revenue analytics)

Field Name	Data Type	Constraint	Description	Comment
TransactionId	**UUID**	Primary Key	Unique identifier for the receipt.	Generated automatically upon a successful checkout.
UserId	**UUID**	Foreign Key	References to UserID in Users table	Identifies which member made the payment.
AmountPaid	**DECIMAL(10, 2)**	Not Null	Amount paid (e.g., 50.00 or 699.00).	Used by the Business Analytics Engine to calculate monthly revenue.
PaymentMethod	**VARCHAR(50)**	Not Null	E.g., Cash, GCash, Card.	Tracks preferred payment methods for gym accounting.
Timestamp	**TIMESTAMP**	Default NOW( )	Date and time the payment is made.	Used to filter and group revenue reports by month or year.

Table 4: Visits (Stores check-in timestamps for attendance and histograms)

Field Name	Data Type	Constraint	Description	Comment

VisitId	**UUID**	Primary Key	Unique identifier for the check-in instance.	Generated upon a successful QR scan.
UserID	**UUID**	Foreign Key	References the members in Users table.	Identifies the member entering the facility.
CheckIn	**TIMESTAMP**	Default NOW( )	The exact time the QR code was scanned to check-in for attendance.	Used by the Analytics Dashboard to generate peak hour histograms.
CheckOut	**TIMESTAMP**	NULL	The exact time the QR code was scanned to check-out for attendance.	Used by the Analytics Dashboard to generate peak hour histograms.

Table 5: Exercises (Defines a specific instance of an exercise a user can perform)

Field Name	Data Type	Constraint	Description	Comment
ExerciseId	**UUID**	Primary Key	Unique identifier for the exercise.	Used as a foreign key in the Routines table.
EquipmentID	**UUID**	Foreign Key	References an Equipment in the Equipments table.	Identifies which equipment is used in the specific exercise.

ExerciseName	**VARCHAR(100)**	Not Null	E.g., "Barbell Squat".	Displayed on the member's daily routine UI.
MuscleGroup	**VARCHAR(50)**	Not Null	Target area (e.g., Chest, Legs, Back).	Used to filter routines (e.g., pulling only 'Chest' exercises for a push day).
MovementType	**VARCHAR(50)**	Not Null	Used for filtering (e.g., Push, Pull).	Relied upon by the Workout Engine logic to build structured splits.

Table 6: Routines (Identifies the status of a Routine and who it's for)

Field Name	Data Type	Constraint	Description	Comment
RoutineID	**UUID**	Primary Key	Unique identifier for this specific logged exercise.	Generated for each exercise assigned to a member on a specific day.
UserID	**UUID**	Foreign Key	References the members in Users table.	Identifies who the routine belongs to.
IsCompleted	**BOOLEAN**	Default FALSE	Tracks if the user actually did it on the gym floor.	Checked off by the user when logging their sets on the mobile portal.
DateAssigned	**DATE**	Not Null	The day this routine was generated for the user.	Ensures the member sees the

Field Name	Data Type	Constraint	Description	Comment
				correct workout for today's date.
DateCompleted	**DATE**	Nullable	The day this routine was accomplished by the user	Can be used to measure the user's performance data.

Table 7: RoutineLists (Keeps track of the list of exercises for each routine generated)

Field Name	Data Type	Constraint	Description	Comment
RoutineID	**UUID**	Primary Key, Foreign Key	Unique identifier for this specific logged exercise.	Generated for each exercise assigned to a member on a specific day.
ExerciseId	**UUID**	Foreign Key	References an exercise in the Exercises table..	Links to the specific movement details.

Table 8: Equipments (Identifies the set of equipments in the gym and its status)

Field Name	Data Type	Constraint	Description	Comment
EquipmentID	**UUID**	Primary Key	Unique identifier for equipment in the gym.	Acts as a foreign key to the Exercises table.
EquipmentStatus	**Varchar(255)**	Not Null	Determines if an equipment is available, under maintenance, or is currently unavailable.	Influences whether or not an exercise is out of service or not.
EquipmentName	**Varchar(100)**	Not Null	The name of the equipment that will be used for the workout.	It informs the system what kind of equipment will

Field Name	Data Type	Constrain t	Description	Comment
				be used for the Exercises table

3.3.2 Security

The WebApp utilizes C# .NET Identity Framework for robust security. User credentials (passwords) are securely hashed before being stored in the Neon.tech PostgreSQL database; plain-text passwords are never stored. The system enforces Role-Based Access Control (RBAC) at the Application Layer, ensuring that API endpoints related to financial data, member management, and business analytics will reject any requests made by users who do not possess the Admin role token.

INDEX

Admin / Gym Owner, 4, 11, 13, 17, 18
Analytics Dashboard, 4, 7, 11, 13, 17, 21
Architecture (N-Tier, System, Monolithic), 4, 7, 13
C# .NET / Backend, 13, 15, 16, 17, 23
Check-In, 8, 15, 21
Database (Neon.tech PostgreSQL), 4, 13, 17, 19, 23
Entity-Relationship Diagram (ERD), 18
Geofencing / Geolocation, 4, 14, 15
Manage Equipment, 12, 18, 22
Manage Members, 12, 17
Member / Customer, 4, 8, 13, 14, 15, 16, 17
Microsoft Azure, 7, 13
Purchase Subscription (E-Commerce), 4, 8, 15
QR Code, 4, 5, 8, 14, 15, 21
Register / Login, 8, 14
Role-Based Access Control (RBAC), 5, 17, 19, 23
Security, 4, 5, 23
Workout Generator / Workout Split, 4, 5, 8, 16, 21